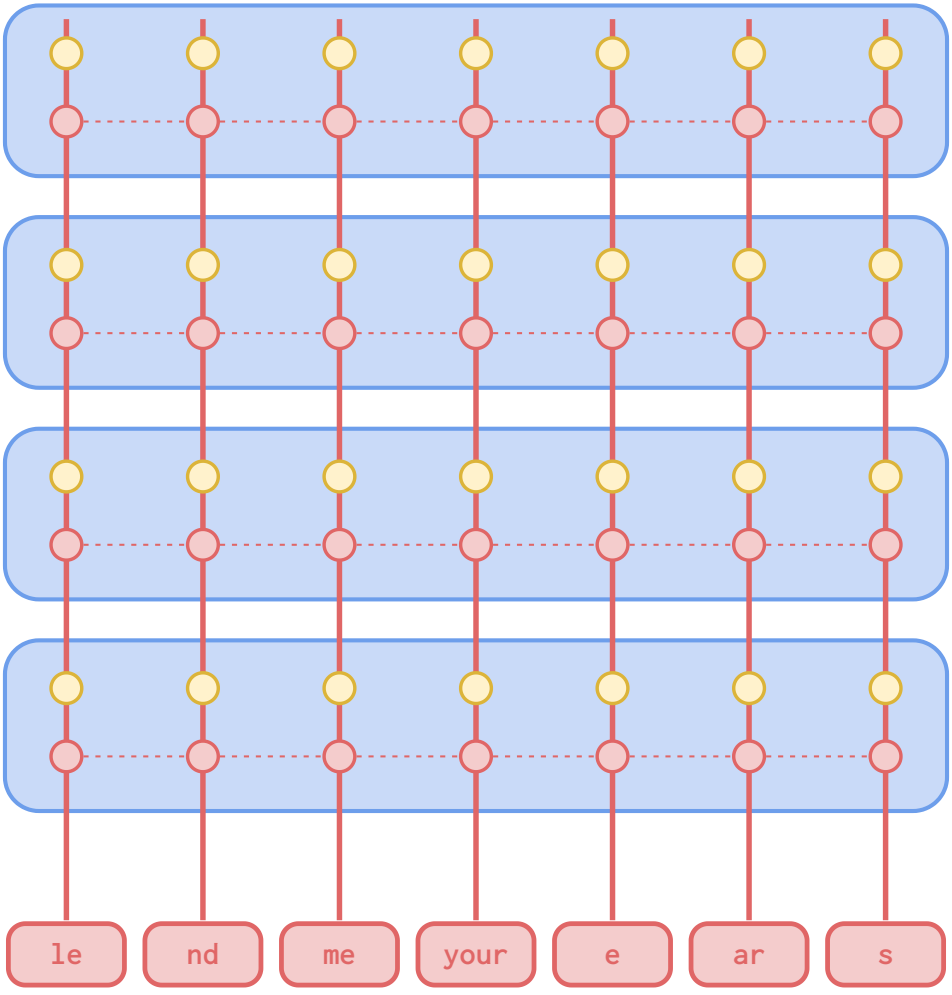


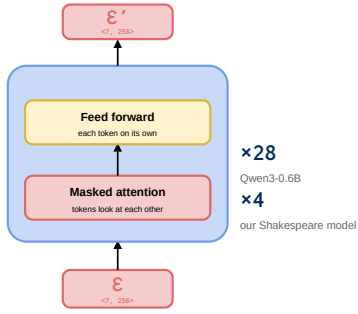
UNIT 01 · CHAPTER 12

Transformer Blocks

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Transformer Blocks

Over the last few chapters we've built the two halves of a transformer. Attention lets every token look back at the tokens before it and pull in what it needs, and the feed-forward network (or a room of experts) then works on each token on its own. Put one after the other and we get a **transformer block**.



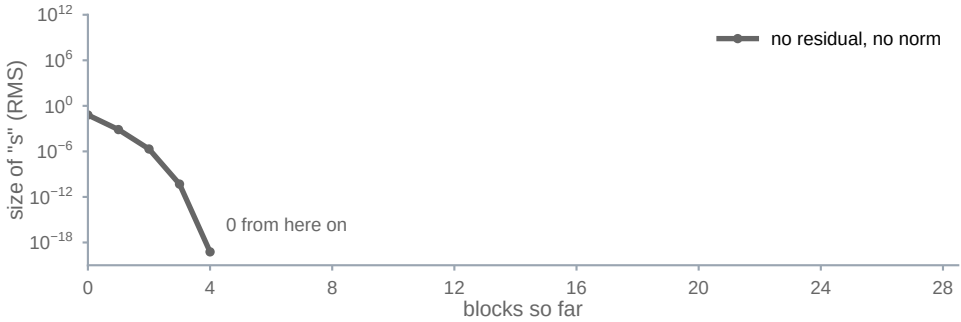
Our running example is the line every Shakespeare student knows, "lend me your ears". irishQwen's Shakespeare tokenizer has only **512** tokens, so the line becomes **7** of them, "le", "nd", " me", " your", " e", "ar" and "s". In the class's Shakespeare model each token is a vector of **256** numbers, so the block takes in a **7** × **256** matrix and **hands back a matrix of exactly the same shape**.

That's what makes blocks easy to stack, the output of one is a perfectly good input for the next. Qwen3-0.6B, the model irishQwen copies, stacks **28** of them, and our Shakespeare model stacks **4**. Unfortunately just stacking them doesn't work, and most of this chapter is about the small pieces that make it work.

Stacking Blocks

To see what goes wrong, we can build **28** blocks the way Qwen3 does (grouped-query attention, then a SwiGLU feed-forward network), fill them with random weights the way irishQwen does before training, and pass "lend me your ears" through them one block at a time.

After each block we'll measure how big the vector for the last token, "s", is. A good measure is its **root mean square** (RMS), roughly the size of a typical number in it. The embedding of "s" starts at an RMS of **0.0626**:

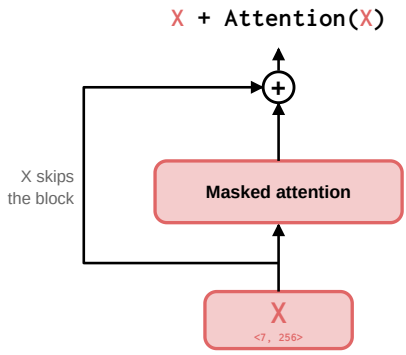


After one block it's **0.000746**, after four it's **5.79** × **10^{-20}**, and from the fifth block on it's exactly **0**. **The token is gone**, and so is anything a later block could have used to predict the next token.

The problem is that every block replaces its input. Whatever a block doesn't pass along is lost for good, and with **28** blocks each passing along a little less, there's nothing left by the top.

Residual Connections

The fix is pretty simple. Instead of replacing the token vectors with the block's output, we **add the block's output to them**:



attention adds to the tokens

$$X \leftarrow X + \text{Attention}(X)$$

and so does the feed-forward network

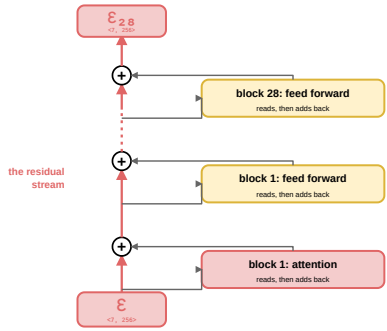
$$X \leftarrow X + \text{FFN}(X)$$

The path that goes around the block is called a **residual connection** (or a skip connection). The block no longer has to pass the token along, it only has to work out what to change about it. Notice that if a block's weights were all `0` it would add nothing, and the token would come out exactly as it went in.

Residual connections come from image classification. Kaiming He and his colleagues at Microsoft used them in 2015 to train a network `152` layers deep, when most networks were closer to `20`, and they've been in pretty much every deep network since.

The Residual Stream

With residual connections, a nice way to think of the whole model is as one stream of token vectors running from the embeddings straight to the top. Each attention layer and each feed-forward network sits beside the stream, reads from it, and adds its result back in:

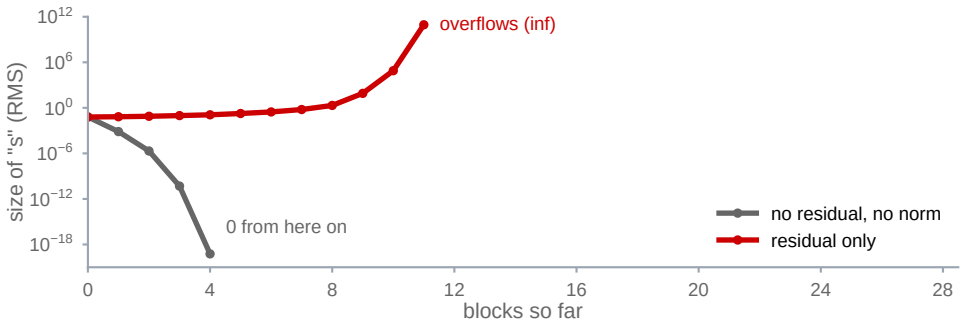


This is called the **residual stream**. After **28** blocks the vector for "s" is its embedding plus **56** additions (an attention and a feed-forward network per block), so the embedding is still in there, along with everything the blocks have added about the tokens around it.

Back in our random blocks, adding instead of replacing does keep "s" alive. After one block the vector for "s" has a cosine similarity of **0.94** with its embedding, and it hasn't shrunk at all (an RMS of **0.0674**). Residual connections also make deep networks much easier to train, for reasons to do with how the gradients flow, which is a story for when we train irishQwen.

Scale Drift

Unfortunately adding has the opposite problem. Here are the same `28` random blocks with residual connections:

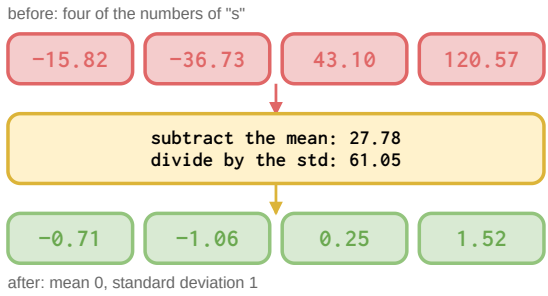


For a few blocks "s" stays about the same size (`0.0674` after one block, `0.121` after four). Then it takes off, `2.08` after eight blocks, `81.1` after nine, `78,200` after ten, and after eleven it's `87.5` billion. At block `12` the numbers are too big for the computer to store and it gives up (it calls them `inf`).

Every addition piles more onto the stream, and nothing ever takes anything away. Worse, the feed-forward network multiplies two copies of its input together (more on that when we get to SwiGLU), so a bigger input gives a **much** bigger output, which makes the next input bigger still. This is **scale drift**: **the size of the numbers depends on how deep we are**, and every block has to cope with inputs of a different size than the block before.

Layer Normalization

So we'd like every block to see its input at a sensible scale, whatever has piled up on the stream. Here are four of the numbers in "s" after nine of those residual-only blocks (the real vector has **256**, but four fit on a page):



Layer normalization (LayerNorm) fixes the scale one token at a time. It subtracts the token's mean, so the numbers are centered on **0**, then divides by their standard deviation, so they have a standard deviation of **1**. Then two learned vectors, γ and β, scale and shift the result, so a layer can still make some numbers bigger than others if that turns out to help:

ε is tiny (1e-5), so we never divide by 0

$$[12.1] \quad y = \frac{x - E[x]}{\sqrt{Var[x] + \epsilon}} * \gamma + \beta$$

LayerNorm in Code

In PyTorch LayerNorm is just a mean, a standard deviation and a division (leaving out γ and β, which start out as ones and zeros anyway):

```
import torch
x = torch.tensor([-15.82, -36.73, 43.10, 120.57])
mean = x.mean()
std = x.std(unbiased=False)
print("mean:", round(mean.item(), 2))
print("std: ", round(std.item(), 2))
y = (x - mean) / std
print("after:", y)
print("new mean:", round(y.mean().item(), 4),
      " new std:", round(y.std(unbiased=False).item(), 4))
```

OUTPUT

```
mean: 27.78
std:  61.05
after: tensor([-0.7142, -1.0567,  0.2510,  1.5200])
new mean: 0.0  new std: 1.0
```

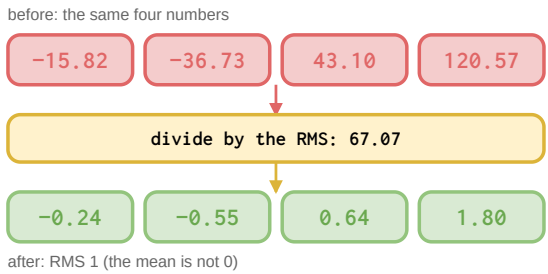
LayerNorm works on each token separately, so it doesn't care how many tokens or prompts there are, and it does exactly the same thing during training and when we use the model.

RMSNorm

Most modern LLMs, Qwen3 included, use a simpler version called **RMSNorm**. It skips subtracting the mean and just divides by the root mean square, the same measure of size we've been plotting:

[12.2] no mean subtracted, and only γ (no β)

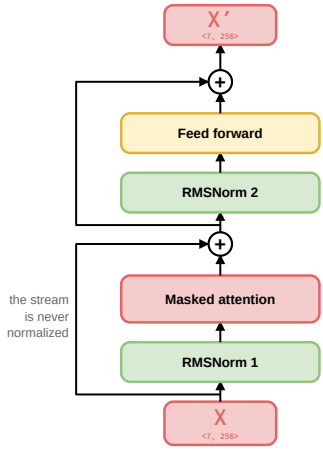
$$y = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} * \gamma$$



The result has an RMS of `1`, but its mean is `0.41`, not `0`, since nothing re-centered it. It turns out that doesn't matter much, the rescaling is what keeps blocks stable, and the re-centering was mostly along for the ride. In return RMSNorm needs one statistic instead of two and has half the weights (γ but no β). irishQwen's `rms_norm.py` does exactly this, with γ starting at all ones.

Pre-Norm

Where the norm goes matters. The original transformer (Vaswani et al., 2017) normalized the stream itself, right after each addition. GPT-2 (2019) moved the norm inside the branch, so it normalizes a copy of the stream on its way into attention or the feed-forward network, and the stream itself is never normalized:



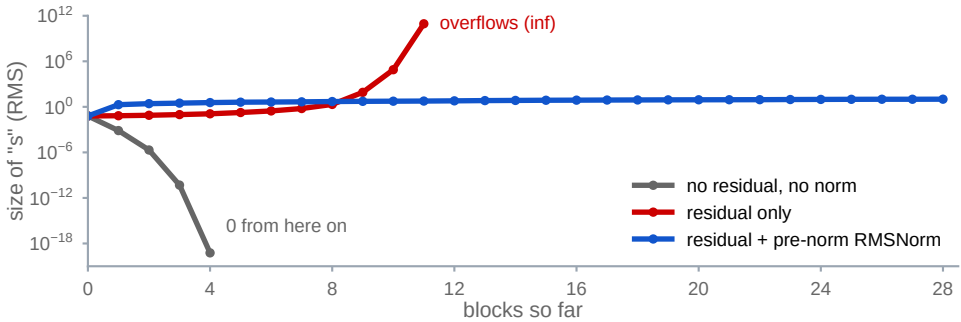
[12.3] the norm only sees a copy

$$X \leftarrow X + \mathrm{Attention}(\mathrm{RMSNorm}(X))$$
$$X \leftarrow X + \mathrm{FFN}(\mathrm{RMSNorm}(X))$$

This is called **pre-norm**, and it's what Qwen3 uses. It keeps both things we wanted. Every block sees an input with an RMS of **1**, however big the stream has grown, and the embedding still rides all the way up the stream untouched. Each block gets its own two RMSNorms (with their own γ), one before attention and one before the feed-forward network.

Pre-Norm at Depth

Here are our **28** random blocks a third time, now with a pre-norm RMSNorm in front of every attention and every feed-forward network:



The stream still grows, since every block still adds to it, but gently, **1.93** after one block, **4.99** after eight and **10.2** after all **28**. That's **a factor of five over 27 blocks**, where residuals alone grew by a factor of **40** billion over three.

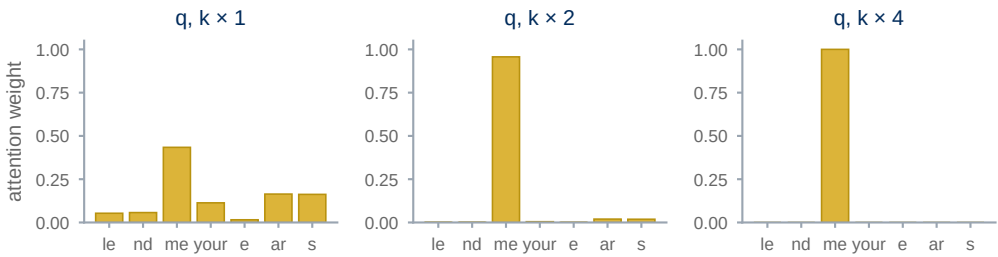
Since the stream itself is never normalized, its last value can be any size. Pre-norm models add one more RMSNorm, the **final RMSNorm**, after the last block and before the output layer that turns each token's vector into scores over the vocabulary, so the output layer always sees the same scale too.

Notice that the first block jumps from **0.0626** to **1.93**. irishQwen's random embeddings start out much smaller than what the blocks add, so at first the blocks drown out the embedding (training sorts this out).

Saturated Attention

Pre-norm keeps the input to attention on a stable scale, but there's one more place numbers can grow, inside attention itself. Each attention score is a dot product of a query and a key, $q \cdot k / \sqrt{d_k}$, so if training makes W_Q and W_K twice as big, every query and key is twice as big and every score is four times as big.

To see what that does, here's a random query (standing in for "s") and seven random keys (standing in for the prompt), with the query and keys made **1**, **2** and **4** times bigger:



At normal size the biggest weight is **0.43** and every token gets a bit. At twice the size one token gets **0.957**, and at four times it gets everything. The softmax has **saturated**: **it has stopped mixing tokens and just picks one**, and nudging the weights a little no longer changes its answer at all, which makes it very hard to train.

QK-Norm

Qwen3's fix is to RMS-normalize every query and every key before the dot product. This is called **QK-Norm**. It works on each head separately, over the head's **32** numbers in our Shakespeare model (**128** in Qwen3-0.6B), with one learned γ for the queries and another for the keys:

the same is done to every key, with its own γ

$$[12.4] \quad q' = \frac{q}{\sqrt{\text{mean}(q^2) + \epsilon}} * \gamma_q$$

In irishQwen QK-Norm happens right after the query and key projections and before RoPE (which we'll get to shortly). The values aren't normalized, since they never go through a softmax. Qwen3 added it and dropped the biases Qwen2 had on its queries, keys and values.

QK-Norm in Code

Here are the same random query and keys as on the last page, made `1`, `2` and `4` times bigger, but normalized before the dot product:

```python
import torch, math
torch.manual_seed(0)
q = torch.randn(32)
K = torch.randn(7, 32)

def rms_norm(x):
    ms = (x * x).mean(dim=-1, keepdim=True)
    return x / torch.sqrt(ms + 1e-6)

for grow in [1, 2, 4]:
    q_n, K_n = rms_norm(grow * q), rms_norm(grow * K)
    scores = q_n @ K_n.T / math.sqrt(32)
    p = torch.softmax(scores, -1).tolist()
    print(grow, [round(v, 3) for v in p])
```

OUTPUT

```
1 [0.036, 0.041, 0.56, 0.091, 0.007, 0.137, 0.127]
2 [0.036, 0.041, 0.56, 0.091, 0.007, 0.137, 0.127]
4 [0.036, 0.041, 0.56, 0.091, 0.007, 0.137, 0.127]
```

The weights are **exactly the same however big q and k get**, since dividing by the RMS undoes any scaling. The scores can no longer grow without limit, so the softmax can't saturate just because the weights grew. (Without the norm, the same query and keys gave `0.434`, `0.957` and `1.0` for the biggest weight.)

Attention Is Order-Blind

There's something we haven't given our blocks at all, the order of the tokens. Attention compares every query with every key, but nothing in that comparison says where the key is. Here's one attention head run on "lend me your ears" and on "lend your me ears" (" me" and " your" swapped):

```python
import torch, math
torch.manual_seed(0)
E = torch.randn(512, 256) * math.sqrt(2 / 512)
Wq, Wk, Wv = [torch.randn(256, 32) / 16 for _ in range(3)]

def head(ids):
    X = E[torch.tensor(ids)]
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    S = Q @ K.T / math.sqrt(32)
    future = torch.ones(len(ids), len(ids)).tril() == 0
    S = S.masked_fill(future, -math.inf)
    return torch.softmax(S, -1) @ V

a = head([316, 269, 320, 350, 335, 290, 115])  # me your
b = head([316, 269, 350, 320, 335, 290, 115])  # your me
print(a[-1, :4])
print(b[-1, :4])
```

OUTPUT

```
tensor([-0.0191, -0.0213, -0.0108, -0.0254])
tensor([-0.0191, -0.0213, -0.0108, -0.0254])
```

The output for "s" is identical. With the causal mask, "s" sees all six tokens before it in both prompts, and since attention only looks at **which** tokens are there and not where, it can't tell the two apart. Attention is **order-blind**: to it, a prompt is a bag of tokens, and **we'll have to put the positions in ourselves**.

Positional Encodings

The original transformer added a vector to each token's embedding saying where it is. Each position `pos` gets its own vector, built from sine and cosine waves:

[12.5] i numbers the pairs of dimensions, d = 256 for us

$$PE(pos, 2i) = \sin\left(pos/10000^{2i/d}\right)$$
$$PE(pos, 2i+1) = \cos\left(pos/10000^{2i/d}\right)$$

These are called **sinusoidal positional encodings**. Our Shakespeare model's context window is **128** tokens, so we need **128** of them:

```python
import torch
d, n = 256, 128        # our model's C and ctx_len
pos = torch.arange(n).float().unsqueeze(1)  # (128, 1)
two_i = torch.arange(0, d, 2).float()       # (128,)
angle = pos / 10000 ** (two_i / d)          # (128, 128)
PE = torch.zeros(n, d)
PE[:, 0::2] = torch.sin(angle)
PE[:, 1::2] = torch.cos(angle)
print(PE[:4, :4])
```

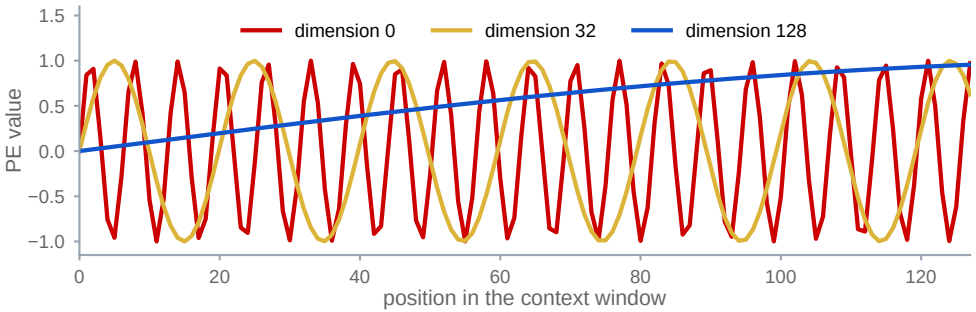
OUTPUT

```
tensor([[ 0.0000,  1.0000,  0.0000,  1.0000],
        [ 0.8415,  0.5403,  0.8020,  0.5974],
        [ 0.9093, -0.4161,  0.9581, -0.2863],
        [ 0.1411, -0.9900,  0.3428, -0.9394]])
```

Position **1** starts with $\sin(1) =$ **0.8415** and $\cos(1) =$ **0.5403**. Adding `PE[:7]` to the **7** × **256** embeddings of "lend me your ears" gives each token its position.

Waves of Different Speeds

Each pair of dimensions is a wave of a different speed. Here are three of the `256` dimensions over the whole context window:



Dimension `0` changes a lot from one position to the next, so it can tell neighbors apart, but it repeats every `6.28` positions, so on its own it can barely tell position `0` from position `44` (`sin(44)` = `0.018`). Dimension `128` hardly changes between neighbors, but it never repeats inside our window. Together the fast and slow waves give every position **its own pattern**, a bit like the hands of a clock (the second hand tells you the second, the hour hand which hour it is).

Sinusoidal encodings have no weights to learn. GPT-2 skipped the waves and simply learned one vector for each of its `1,024` positions, as one more embedding table. Either way, the position is added to the token, so **every block downstream gets token and position mixed together** in the same `256` numbers.

Relative Position

For predicting the next word, where a token sits matters much less than how far apart tokens are (" e" right after " your" is a hint wherever the pair is). Let's slide " e" (the query) and " your" (the key), always one apart, along the context window with sinusoidal encodings added:

```python
import torch, math
torch.manual_seed(0)
E = torch.randn(512, 256) * math.sqrt(2 / 512)
Wq = torch.randn(256, 32) / 16
Wk = torch.randn(256, 32) / 16
pos = torch.arange(128).float().unsqueeze(1)
angle = pos / 10000 ** (torch.arange(0, 256, 2) / 256)
PE = torch.zeros(128, 256)
PE[:, 0::2] = torch.sin(angle)
PE[:, 1::2] = torch.cos(angle)

e, your = E[335], E[350]       # " e" and " your"
e = e / e.pow(2).mean().sqrt()     # RMSNorm, y = 1
your = your / your.pow(2).mean().sqrt()
for p in [0, 10, 50, 100]:    # " your" at p, " e" at p+1
    q = (e + PE[p + 1]) @ Wq
    k = (your + PE[p]) @ Wk
    print(p, round((q @ k).item() / math.sqrt(32), 3))
```

OUTPUT

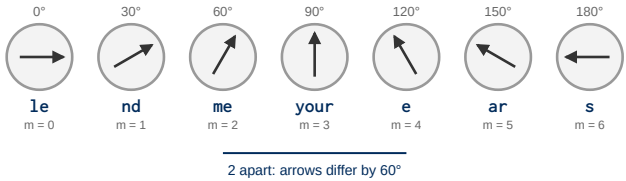
```
0 -1.26
10 -0.24
50 0.724
100 0.03
```

Same tokens, same distance, but the score goes from **−1.26** to **0.724** depending on where they sit. **The score depends on absolute positions**, not just on the distance.

Rotations

That's because the positions are mixed into the token vectors before the queries and keys are computed (over all `127` places the score ranges from `−2.84` to `1.83`). We'd rather the distance was built in.

Think about two arrows on a clock face. If we turn both of them by the same angle, the angle between them doesn't change. So instead of adding a position, **we turn each token's query and key by an angle that depends on its position**.



In two dimensions, turning a vector by an angle $m\theta$ is a multiplication by a **rotation matrix**:

m is the token's position

[12.6] $$R(m) = \begin{bmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{bmatrix}$$

The dot product of two arrows only depends on their lengths and the angle between them, so if a query at position `4` is turned by `120`° and a key at position `3` by `90`°, the angle between them grows by `30`°, exactly as it would at positions `51` and `50`. **The score only sees the distance, one position**.

Rotary Positional Embeddings

Our queries and keys have **32** numbers per head, not two, so RoPE splits them into **16** pairs and turns each pair by its own angle. Pair i turns by θ_i per position, with the same kind of fast and slow speeds as the sinusoidal waves:

[12.7] $i = 0 \ldots 15$, $d = 32$ numbers per head

$$\theta_i = \text{base}^{-2i/d}$$

[12.8] one 2×2 rotation per pair, zeros everywhere else

$$R(m) = \begin{bmatrix} \cos m\theta_0 & -\sin m\theta_0 & & \\ \sin m\theta_0 & \cos m\theta_0 & & \\ & & \ddots & \\ & & & R_{15} \end{bmatrix}$$

This is **Rotary Positional Embeddings** (RoPE), from Jianlin Su and his colleagues in 2021. The query and key of the token at position m are both multiplied by $R(m)$ just before the dot product.

A few details matter for irishQwen. **Only the queries and keys are rotated**, never the values, since position only matters for deciding who looks at whom. RoPE has no weights to learn, so one table of sines and cosines is shared by every block. Our Shakespeare model uses a base of **10,000**, Qwen3-0.6B **1,000,000** (slower pairs, for its much longer context). irishQwen pairs number i with $i + 16$ rather than its neighbor (quicker to compute, same idea).

RoPE in Code

The same experiment with RoPE, building `R(m)` one pair at a time, one position apart and three apart:

```python
import torch, math
torch.manual_seed(0)
E = torch.randn(512, 256) * math.sqrt(2 / 512)
Wq = torch.randn(256, 32) / 16
Wk = torch.randn(256, 32) / 16
theta = 10000 ** (-torch.arange(0, 32, 2) / 32)

def R(m):            # rotate pair j by angle m * theta_j
    Rm = torch.zeros(32, 32)
    for j in range(16):
        c = math.cos(m * theta[j])
        s = math.sin(m * theta[j])
        Rm[2*j:2*j+2, 2*j:2*j+2] = torch.tensor([[c, -s],
                                                 [s, c]])
    return Rm

e, your = E[335], E[350]         # " e" and " your"
q = e / e.pow(2).mean().sqrt() @ Wq          # RMSNorm
k = your / your.pow(2).mean().sqrt() @ Wk
for p in [0, 10, 50, 100]:
    gap1 = (R(p + 1) @ q) @ (R(p) @ k) / math.sqrt(32)
    gap3 = (R(p + 3) @ q) @ (R(p) @ k) / math.sqrt(32)
    print(p, round(gap1.item(), 3), round(gap3.item(), 3))
```

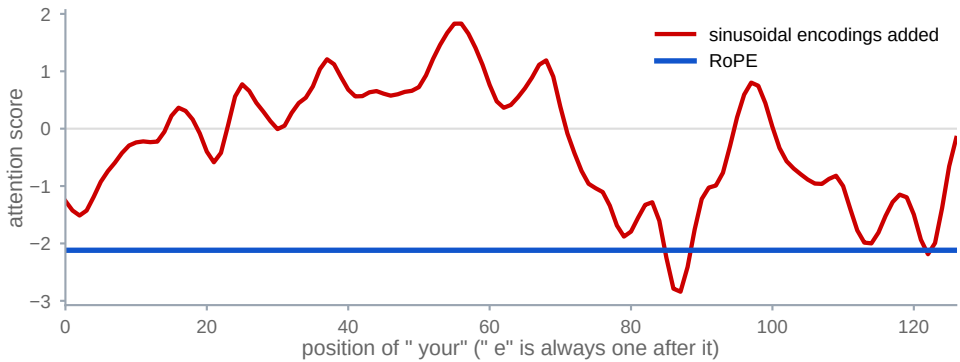
OUTPUT

```
0 -2.119 -2.273
10 -2.119 -2.273
50 -2.119 -2.273
100 -2.119 -2.273
```

One position apart the score is **−2.119** wherever the pair is, and three apart it's **−2.273** wherever it is. **The score depends on what the tokens are and how far apart they are, and nothing else**.

RoPE vs. Sinusoidal Encodings

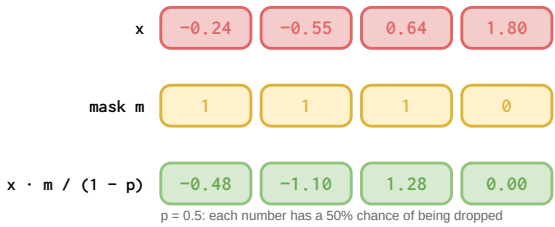
Here's the score between " e" and " your", one position apart, at every place in the context window, with sinusoidal encodings added and with RoPE:



The sinusoidal score wanders with the position, while the RoPE score is a flat line. To your author this is one of the trickiest ideas of the last few years, the positions never touch the token vectors, and yet attention ends up knowing exactly how far apart every pair of tokens is.

Dropout

One last piece of the older recipe is something Qwen3 leaves out. **Dropout** sets random numbers in a layer's output to `0` during training, a fresh random set at every step, so the network can't come to depend on any single number. To keep the average the same it scales up the numbers it keeps, dividing by $1 - p$, where p is the chance of dropping each one:



The next step gets a new mask, so a different set of numbers is dropped, and the numbers that survive are doubled (since p = `0.5`). Dropout is **only used during training**, when we use the model it does nothing at all.

GPT-2 used dropout with p = `0.1` in three places, on the embeddings, on the attention weights, and on the output of every attention layer and feed-forward network before it's added to the stream.

No Dropout

Dropout is a way to fight overfitting, a network memorizing its training data instead of learning something that works on new text. It forces the network to spread what it knows over many numbers, so that no single one can memorize a particular sentence.

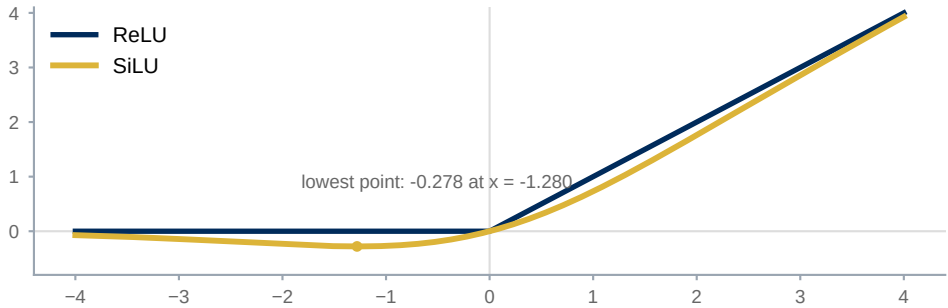
Modern LLMs are trained on so much text, and such varied text, that they usually see most of it only once. A model can't memorize a sentence it has seen once, so **overfitting is much less of a concern**, and dropout would only throw away part of the signal at every step. Qwen3 doesn't use it anywhere, and neither does irishQwen.

Our Shakespeare model is the exception to the rule. It trains for five passes over a `5.4` MB corpus, which is exactly the setting where overfitting can happen. That's why the training script holds back the last `5`% of the corpus as validation text, if the loss on those lines starts going up while the training loss keeps going down, the model has started memorizing Shakespeare.

SiLU

There's one more difference inside the feed-forward network, the activation function. Instead of ReLU, Qwen3 uses **SiLU** (also called Swish), which multiplies **x** by its own sigmoid:

$$[12.9]\quad \mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$



For big positive inputs SiLU is almost exactly ReLU, and for very negative ones it's almost `0`. In between it's smooth, with no corner at `0`, and it dips a little below `0` (to `−0.278` at x = `−1.278`).

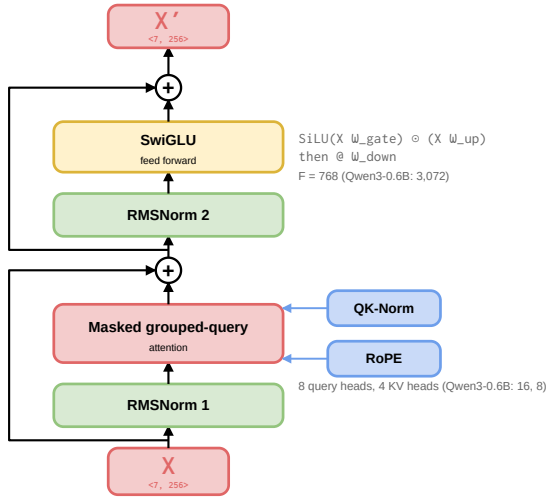
Qwen3 uses it inside a **SwiGLU** feed-forward network, where a "gate" branch goes through SiLU and decides how much of an "up" branch to let through:

$$[12.10]\quad \mathrm{FFN}(X) = \big(\mathrm{SiLU}(X W_{\mathrm{gate}}) \odot X W_{\mathrm{up}}\big) W_{\mathrm{down}}$$

$\odot$ multiplies number by number, no biases

The Qwen3 Block

Putting it all together, here's one Qwen3 block, exactly as irishQwen's `qwen_transformer.py` builds it:



Compared with IrishChat, the GPT-2-style model the class built earlier, every one of the changes is in this picture:

	IrishChat (GPT-2-style)	Qwen3 / irishQwen
attention	multi-head	grouped-query, with QK-Norm
activation	ReLU	SiLU, in a SwiGLU network
normalization	LayerNorm	RMSNorm (pre-norm)
position	added to the embeddings	RoPE on queries and keys
dropout	yes	none

A Block in Code

Here are the pieces of a block in PyTorch, sized like our Shakespeare model and filled with random weights the way irishQwen starts out. The attention is grouped-query (8 query heads sharing 4 key and value heads), and the feed-forward network is SwiGLU. To keep it short we leave out QK-Norm and RoPE (irishQwen's versions are part of building it). First the sizes, the embeddings and the weights:

```python
import torch, math
torch.manual_seed(0)
C, H, H_kv, Dh, F = 256, 8, 4, 32, 768
ids = torch.tensor([316, 269, 320, 350, 335, 290, 115])
T = len(ids)                    # "lend me your ears"
E = torch.randn(512, C) * math.sqrt(2 / 512)

def rms_norm(X):
    ms = (X * X).mean(dim=-1, keepdim=True)
    return X / torch.sqrt(ms + 1e-6)

def w(n_in, n_out, gain):
    s = math.sqrt(gain / n_in)
    return torch.randn(n_in, n_out) * s

def new_block():
    return {"Wq": w(C, H * Dh, 1),
            "Wk": w(C, H_kv * Dh, 1),
            "Wv": w(C, H_kv * Dh, 1),
            "Wo": w(H * Dh, C, 1),
            "Wg": w(C, F, 2), "Wu": w(C, F, 2),
            "Wd": w(F, C, 2)}
print(E[ids].shape)
```

OUTPUT

```
torch.Size([7, 256])
```

Attention and SwiGLU in Code

Then the two halves of a block, grouped-query attention with the causal mask, and the SwiGLU feed-forward network, and 28 blocks of random weights:

```
def attention(X, b):
    Q = (X @ b["Wq"]).view(T, H, Dh).transpose(0, 1)
    K = (X @ b["Wk"]).view(T, H_kv, Dh).transpose(0, 1)
    V = (X @ b["Wv"]).view(T, H_kv, Dh).transpose(0, 1)
    K = K.repeat_interleave(H // H_kv, dim=0)
    V = V.repeat_interleave(H // H_kv, dim=0)
    S = Q @ K.transpose(1, 2) / math.sqrt(Dh)
    future = torch.ones(T, T).tril() == 0
    S = S.masked_fill(future, -math.inf)
    ctx = torch.softmax(S, -1) @ V
    ctx = ctx.transpose(0, 1).reshape(T, H * Dh)
    return ctx @ b["Wo"]

def swiglu(X, b):
    G = X @ b["Wg"]
    A = G * torch.sigmoid(G) * (X @ b["Wu"])
    return A @ b["Wd"]

blocks = [new_block() for _ in range(28)]
print(attention(E[ids], blocks[0]).shape)
```

OUTPUT

```
torch.Size([7, 256])
```

Stacking Blocks in Code

A block can be put together three ways, plain, residual only, or residual with pre-norm. This is the code behind the three curves earlier in the chapter (it prints the size of "s" after blocks **1**, **4**, **8**, **12** and **28**):

```
def block(X, b, mode):
    if mode == "plain":
        return swiglu(attention(X, b), b)
    if mode == "residual":
        X = X + attention(X, b)
        return X + swiglu(X, b)
    X = X + attention(rms_norm(X), b)    # pre-norm
    return X + swiglu(rms_norm(X), b)

for mode in ["plain", "residual", "pre-norm"]:
    X, sizes = E[ids], []
    for b in blocks:
        X = block(X, b, mode)
        sizes.append(X[-1].pow(2).mean().sqrt().item())
        if sizes[-1] > 1e9:          # overflowing
            break
    sizes += [math.inf] * (28 - len(sizes))
    picks = [sizes[n - 1] for n in (1, 4, 8, 12, 28)]
    print(f"{mode:9}", [f"{s:.3g}" for s in picks])
```

OUTPUT

```
plain     ['0.000746', '5.79e-20', '0', '0', '0']
residual  ['0.0674', '0.121', '2.08', 'inf', 'inf']
pre-norm  ['1.93', '3.64', '4.99', '6.17', '10.2']
```

Only the last version, `pre-norm`, is a Qwen3 block. Its two lines are the skeleton of `qwen_transformer.py`'s forward pass.

Counting Weights

Almost all of a model's weights live in its blocks. Each Qwen3-0.6B block has four attention matrices (the key and value ones half as wide, thanks to grouped-query attention), three feed-forward matrices of `1,024` × `3,072`, and a few small norm vectors:

```
C, H, H_kv, Dh, F = 1024, 16, 8, 128, 3072
V, L = 151936, 28                  # Qwen3-0.6B
attn = C * H * Dh + 2 * C * H_kv * Dh + H * Dh * C
ffn = 3 * C * F                    # gate, up and down
norms = 2 * C + 2 * Dh             # 2 RMSNorms + QK-Norm
block = attn + ffn + norms
print("one block:  ", f"{block:,}")
print("28 blocks:  ", f"{L * block:,}")
print("embeddings: ", f"{V * C:,}")
print("total:      ", f"{L * block + V * C + C:,}")
```

OUTPUT

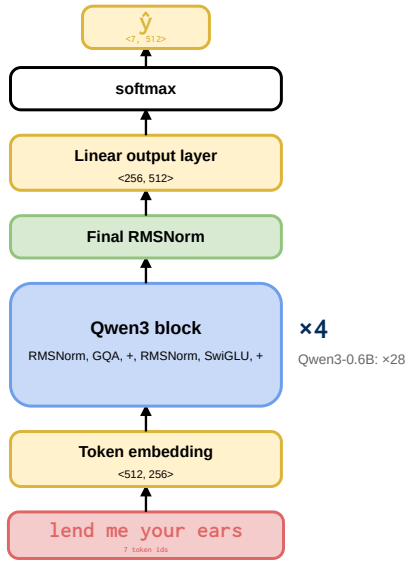
```
one block:   15,730,944
28 blocks:   440,466,432
embeddings:  155,582,464
total:       596,049,920
```

Each block has about `15.7` million weights, `28` blocks have `440` million, and the embedding table (which Qwen3-0.6B also uses as its output layer) adds `156` million, for `596` million in all, the "0.6B" in the name. Notice how little the norms add, `2,304` numbers per block, and RoPE adds nothing.

Our Shakespeare model is the same design, much smaller, `787,008` weights per block, about `3.4` million in all.

From Blocks to irishQwen

We now have every piece of irishQwen. The token ids go into the embedding table, the token vectors run up the residual stream through the blocks, a final RMSNorm puts them back on a common scale, and the output layer and a softmax turn each one into a probability for every token in the vocabulary:



So far every block has had random weights, so the probabilities at the top mean nothing yet. In the next chapter we'll put these pieces together into one model, count its weights, and **train it on Shakespeare**.